



Decidability (4.1 and 4.2)

COT 4210 Discrete Structures II
Summer 2025
Department of Computer Science
Dr. Steinberg

Acceptance of Regular Languages

- Consider a language encoding the question of whether a given DFA accepts a given string.
- $A_{\text{DFA}} = \{ \langle D, s \rangle \mid D \text{ is a DFA that accepts } s \}$.
- Observe that:
 - DFAs always finish computing in $|s|$ steps, with s being the input.
 - A DFA's computational method is an algorithm.

Decidability of Regular Languages

- Consider a language encoding the question of whether a given DFA accepts a given string.
- $A_{\text{DFA}} = \{ \langle D, s \rangle \mid D \text{ is a DFA that accepts } s \}$.
- Observe that:
 - DFAs always finish computing in $|s|$ steps, with s being the input.
 - A DFA's computational method is an algorithm.
- **On input $\langle D, s \rangle$:**
 1. Simulate $D = (Q_D, \Sigma_D, \delta_D, q_{0D}, F_D)$ on input s .
 2. If the simulation ends on an accepting state, accept. Otherwise, reject.

Decidability of Regular Languages

- Consider a language encoding the question of whether a given DFA accepts a given string.
- $A_{DFA} = \{ \langle D, s \rangle \mid D \text{ is a DFA that accepts } s \}$.
- Observe that:
 - DFAs always finish computing in $|s|$ steps, with s being the input.
 - A DFA's computational method is an algorithm.
- **On input $\langle D, s \rangle$:**
 1. Simulate $D = (Q_D, \Sigma_D, \delta_D, q_{0D}, F_D)$ on input s .
 2. If the simulation ends on an accepting state, accept. Otherwise, reject.
- Conclusion: Determining whether a DFA accepts a string is Turing-decidable (and hence, so is determining whether any regular language contains any string).

Emptiness of Regular Languages

- It's useful to be able to figure out if a regular language is empty.
- $E_{\text{DFA}} = \{ \langle \mathbf{D} \rangle \mid \mathbf{D} \text{ is a DFA and } L(\mathbf{D}) = \emptyset \}$.
- Observe that:
 - Any regular language can be recognized by a DFA.
 - A DFA accepts some string if and only if it's possible to get to an accept state from the start state.

Emptiness of Regular Languages

- It's useful to be able to figure out if a regular language is empty.
- $E_{DFA} = \{ \langle D \rangle \mid D \text{ is a DFA and } L(D) = \emptyset \}$.
- Observe that:
 - Any regular language can be recognized by a DFA.
 - A DFA accepts some string if and only if it's possible to get to an accept state from the start state.
- **On input $\langle D \rangle$:**
 1. Mark the start state of D .
 2. Repeat until we don't mark any new states:
 - a. Iterate through all marked states. For each state q :
 - Mark all states that we can transition to from q .
 3. If an accept state is marked, reject. Otherwise, accept.
- Conclusion: Whether or not a regular language is empty is decidable.

Equivalence of Regular Languages

- It's obviously useful to be able to figure out whether two regular languages are equivalent. It turns out that this is decidable as well – in fact, it's fairly easy.
- $EQ_{DFA} = \{ \langle D, E \rangle \mid D \text{ and } E \text{ are DFAs and } L(D) = L(E) \}$.

Equivalence of Regular Languages

- It's obviously useful to be able to figure out whether two regular languages are equivalent. It turns out that this is decidable as well – in fact, it's fairly easy.
- $\mathbf{EQ}_{\text{DFA}} = \{ \langle \mathbf{D}, \mathbf{E} \rangle \mid \mathbf{D} \text{ and } \mathbf{E} \text{ are DFAs and } L(\mathbf{D}) = L(\mathbf{E}) \}$.
 - Recall that the *symmetric difference* $A \Delta B$ of two sets is defined as $(A - B) \cup (B - A)$.
 - The symmetric difference is “everything that's in B that isn't in A, and everything that's in A that isn't in B.”
 - Put that way, it's obvious that two sets are equal iff their symmetric difference is empty.
 - We already know that union and set difference preserve regularity; symmetric difference must as well.

Equivalence of Regular Languages

- It's obviously useful to be able to figure out whether two regular languages are equivalent. It turns out that this is decidable as well – in fact, it's fairly easy.
- $\text{EQ}_{\text{DFA}} = \{ \langle \mathbf{D}, \mathbf{E} \rangle \mid \mathbf{D} \text{ and } \mathbf{E} \text{ are DFAs and } L(\mathbf{D}) = L(\mathbf{E}) \}$.
 - Recall that the *symmetric difference* $A \Delta B$ of two sets is defined as $(A - B) \cup (B - A)$.
 - The symmetric difference is “everything that's in B that isn't in A, and everything that's in A that isn't in B.”
 - Put that way, it's obvious that two sets are equal iff their symmetric difference is empty.
 - We already know that union and set difference preserve regularity; symmetric difference must as well.
 - All that remains is to observe that *two languages are equal if and only if their symmetric difference is empty* - and we just showed that we can decide if a regular language is empty!

Equivalence of Regular Languages

- It's obviously useful to be able to figure out whether two regular languages are equivalent. It turns out that this is decidable as well – in fact, it's fairly easy.
- $EQ_{DFA} = \{ \langle D, E \rangle \mid D \text{ and } E \text{ are DFAs and } L(D) = L(E) \}$.
 - Recall that the *symmetric difference* $A \Delta B$ of two sets is defined as $(A - B) \cup (B - A)$.
 - The symmetric difference is “everything that's in B that isn't in A, and everything that's in A that isn't in B.”
 - Put that way, it's obvious that two sets are equal iff their symmetric difference is empty.
 - We already know that union and set difference preserve regularity; symmetric difference must as well.
 - All that remains is to observe that *two languages are equal if and only if their symmetric difference is empty* - and we just showed that we can decide if a regular language is empty!
- On input $\langle D_A, D_B \rangle$:
 1. Construct D_C , a DFA that recognizes the symmetric difference of the languages of D_A and D_B .
 2. Determine if $L(D_C)$ is empty. Accept if so, reject if not.

Decidability of CFLs

The easy way to *recognizability* is to take a CFG **G** and just keep iterating on it – try all possible derivations of **G** and accept if any of them ever generate *s*.

We don't want recognizability, we want decidability.

Decidability of CFLs

The easy way to *recognizability* is to take a CFG **G** and just keep iterating on it – try all possible derivations of **G** and accept if any of them ever generate *s*.

We don't want recognizability, we want decidability.

Think about a new grammar **G** in Chomsky Normal Form. **Every** rule in a CNF grammar has one of two forms:

- $A \rightarrow BC$
- $A \rightarrow a$

Now we make two observations.

Decidability of CFLs

The easy way to *recognizability* is to take a CFG **G** and just keep iterating on it – try all possible derivations of **G** and accept if any of them ever generate s .

We don't want recognizability, we want decidability.

Think about a new grammar **G** in Chomsky Normal Form. **Every** rule in a CNF grammar has one of two forms:

- $A \rightarrow BC$
- $A \rightarrow a$

Now we make two observations.

1. Given s with $|s| = n$, there are exactly $2n - 1$ steps to derive s :
 - $n - 1$ to grow out to n variables,
 - n to convert them all to terminals.
2. There are a finite number of derivations with $2n - 1$ steps in any CFG.

Decidability of CFLs

The easy way to *recognizability* is to take a CFG **G** and just keep iterating on it – try all possible derivations of **G** and accept if any of them ever generate s .

We don't want recognizability, we want decidability.

Think about a new grammar **G** in Chomsky Normal Form. **Every** rule in a CNF grammar has one of two forms:

- $A \rightarrow BC$
- $A \rightarrow a$

Now we make two observations.

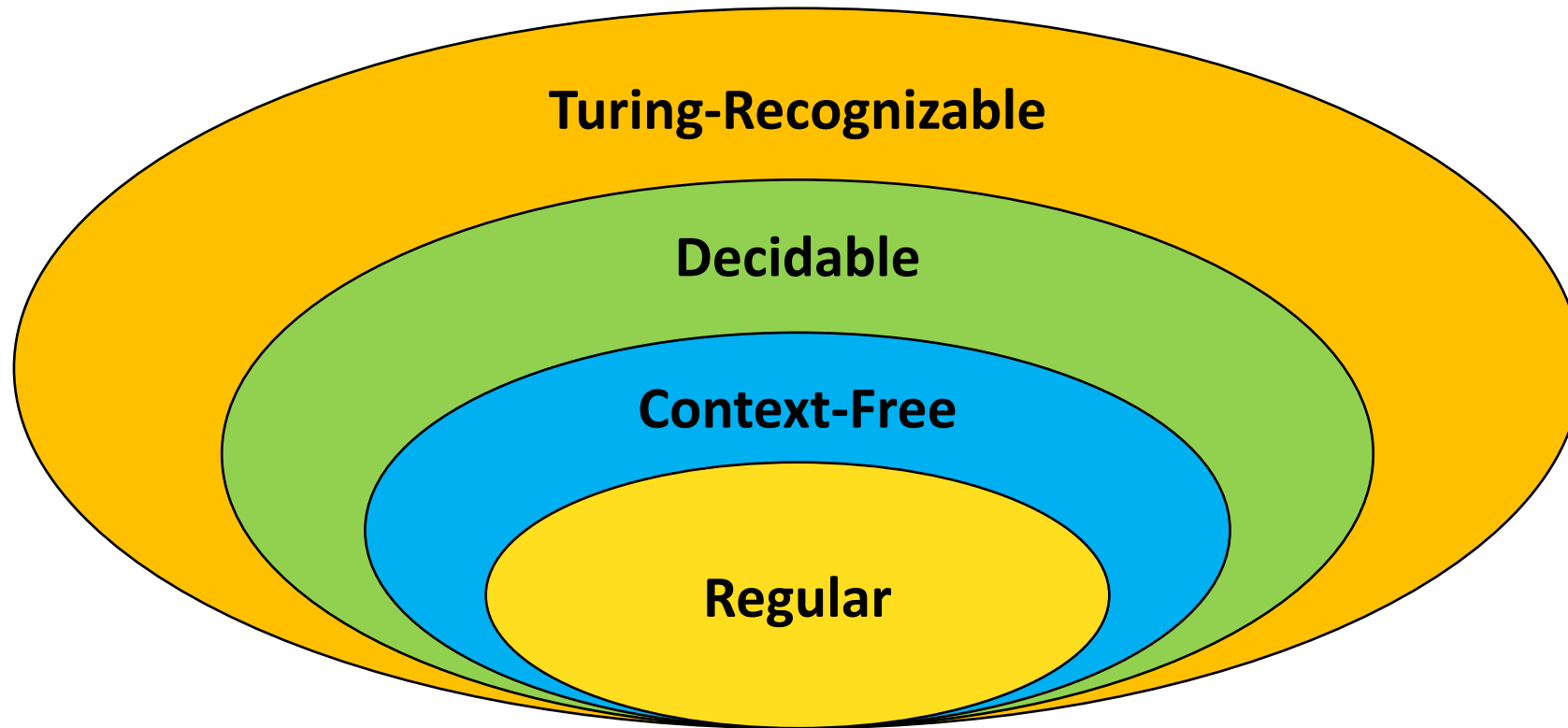
1. Given s with $|s| = n$, there are exactly $2n - 1$ steps to derive s :
 - $n - 1$ to grow out to n variables,
 - n to convert them all to terminals.
2. There are a finite number of derivations with $2n - 1$ steps in any CFG.

On input $\langle G, s \rangle$, with $|s| = n$:

1. Find $n = |s|$.
2. Convert **G** to CNF.
3. Iterate through all derivations in **G** with $2n - 1$ steps.
 - If any of them generate s , accept.
4. Otherwise, reject.

Emptiness of CFLs

- This works a lot like the one for regular languages did, but we work backwards.
- **On input $\langle G \rangle$:**
 1. Mark all the terminal symbols in G .
 2. Repeat until no new variables get marked:
 - a. Iterate over every rule $A \rightarrow R$:
 - If every symbol in R has been marked, mark A .
 3. If the start variable is marked, reject; otherwise, accept.
- Equivalence, unlike with regular languages, is *not* decidable – CFLs aren't closed under complement or intersection.



Something that does not decide. Undecidable

A_{TM} is undecidable. Let's create a Turing Machine U that recognizes A_{TM} .

Something that does not decide. Undecidable

A_{TM} is undecidable. Let's create a Turing Machine U that recognizes A_{TM} .

On input $\langle M, s \rangle$:

Something that does not decide. Undecidable

A_{TM} is undecidable. Let's create a Turing Machine U that recognizes A_{TM} .

On input $\langle M, s \rangle$:

1. Simulate M on input s .

Something that does not decide. Undecidable

A_{TM} is undecidable. Let's create a Turing Machine U that recognizes A_{TM} .

On input $\langle M, s \rangle$:

1. Simulate M on input s .
2. If M ever enters its accept state, accept. If M ever enters its reject state, reject.

Something that does not decide. Undecidable

A_{TM} is undecidable. Let's create a Turing Machine U that recognizes A_{TM} .

On input $\langle M, s \rangle$:

1. Simulate M on input s .
2. If M ever enters its accept state, accept. If M ever enters its reject state, reject.

This machine loops on input $\langle M, s \rangle$ if M loops on s , which is why the machine does not decide A_{TM} .

Something that does not decide. Undecidable

A_{TM} is undecidable. Let's create a Turing Machine U that recognizes A_{TM} .

On input $\langle M, s \rangle$:

1. Simulate M on input s .
2. If M ever enters its accept state, accept. If M ever enters its reject state, reject.

This machine loops on input $\langle M, s \rangle$ if M loops on s , which is why the machine does not decide A_{TM} .

If there was some magical way to determine M was not halting on s , it could reject, but that doesn't exist!!!

Something that does not decide. Undecidable

A_{TM} is undecidable. Let's create a Turing Machine U that recognizes A_{TM} .

On input $\langle M, s \rangle$:

1. Simulate M on input s .
2. If M ever enters its accept state, accept. If M ever enters its reject state, reject.

This machine loops on input $\langle M, s \rangle$ if M loops on s , which is why the machine does not decide A_{TM} .

If there was some magical way to determine M was not halting on s , it could reject, but that doesn't exist!!!

This particular Turing Machine we just designed is an example of the Universal Turing Machine.

Something that does not decide. Undecidable

A_{TM} is undecidable. Let's create a Turing Machine U that recognizes A_{TM} .

On input $\langle M, s \rangle$:

1. Simulate M on input s .
2. If M ever enters its accept state, accept. If M ever enters its reject state, reject.

This machine loops on input $\langle M, s \rangle$ if M loops on s , which is why the machine does not decide A_{TM} .

If there was some magical way to determine M was not halting on s , it could reject, but that doesn't exist!!!

What we should notice is that recognizers are extremely powerful. More powerful than deciders!

A Brief Digression Into Infinity

At Least, By Comparison

Some Infinite Sets

- Let's consider a few infinite sets:
 - $\mathbb{N}$, the set of natural numbers.
 - $\mathbb{Z}$, the set of integers.
 - $\mathbb{Q}$, the set of rational numbers.
 - $\mathbb{R}$, the set of real numbers.
- **All but one of these are actually the same size.**
 - How do we know?

The Size of Infinity

- First we need a definition of size that even makes sense when we're talking about infinite sets. Recall injections, surjections, and bijections. A function is:
 - *One-to-one* (an *injection*) if it maps every element of the range from at most one element of the domain.
 - *Onto* (a *surjection*) if it maps every element of the range from at least one element of the domain.
 - A *correspondence* (*bijection*) if every element of the range is mapped by exactly one element of the domain.
- **Two sets S and T – including infinite sets – are of the same size if there is a correspondence between S and T .**

The Easy One: Integers

- So let's create a correspondence between $\mathbb{N}$ and $\mathbb{Z}$:

$\mathbb{N}$	$\mathbb{Z}$
0	0
1	1
2	-1
3	2
4	-2
...	...

- ...and so on. That one's pretty easy.

The Hard One: Rationals

- We can also create a correspondence between $\mathbb{N}$ and $\mathbb{Q}$, but it takes more thought.
- First, lay out the rational numbers on a grid, as shown to the right.

1/1	1/2	1/3	1/4	1/5	...
2/1	2/2	2/3	2/4	2/5	
3/1	3/2	3/3	3/4	3/5	
4/1	4/2	4/3	4/4	4/5	
5/1	5/2	5/3	5/4	5/5	
...					

The Hard One: Rationals

- We can also create a correspondence between $\mathbb{N}$ and $\mathbb{Q}$, but it takes more thought.
- First, lay out the rational numbers on a grid, as shown to the right.
- Now go *diagonally* through the grid, starting from the top left.

1/1	1/2	1/3	1/4	1/5	...
2/1	2/2	2/3	2/4	2/5	
3/1	3/2	3/3	3/4	3/5	
4/1	4/2	4/3	4/4	4/5	
5/1	5/2	5/3	5/4	5/5	
...					

The Hard One: Rationals

- We can also create a correspondence between $\mathbb{N}$ and $\mathbb{Q}$, but it takes more thought.
- First, lay out the rational numbers on a grid, as shown to the right.
- Now go *diagonally* through the grid, starting from the top left.
- Ignore numbers we've already hit, so that we don't create duplicates and break the bijective properties.

1/1	1/2	1/3	1/4	1/5	...
2/1	2/2	2/3	2/4	2/5	
3/1	3/2	3/3	3/4	3/5	
4/1	4/2	4/3	4/4	4/5	
5/1	5/2	5/3	5/4	5/5	
...					

The Hard One: Rationals

- We can also create a correspondence between $\mathbb{N}$ and $\mathbb{Q}$, but it takes more thought.
- First, lay out the rational numbers on a grid, as shown to the right.
- Now go *diagonally* through the grid, starting from the top left.
- Ignore numbers we've already hit, so that we don't create duplicates and break the bijective properties.
- And that's it. Using **Cantor's diagonalization method**, we've created a correspondence between $\mathbb{N}$ and $\mathbb{Q}$.

1/1	1/2	1/3	1/4	1/5	...
2/1	2/2	2/3	2/4	2/5	
3/1	3/2	3/3	3/4	3/5	
4/1	4/2	4/3	4/4	4/5	
5/1	5/2	5/3	5/4	5/5	
...					

Counting to Infinity

- A **countably infinite** set is a set that is the same size as $\mathbb{N}$; that is, a set for which we can make a correspondence to $\mathbb{N}$.
 - A **countable** set is any set that's either finite or countably infinite.
 - We don't usually bother talking about this stuff with finite sets, though, so you'll see a lot of referring to countably infinite sets as just "countable".
- Sadly, not all infinite sets are countable.

The Impossible One: Reals

- Let f be a correspondence from $\mathbb{N}$ to $\mathbb{R}$. So each number $f(n)$ must be a real number.

The Impossible One: Reals

- Let f be a correspondence from $\mathbb{N}$ to $\mathbb{R}$. So each number $f(n)$ must be a real number.
- Now construct a number x , between 0 and 1, with an infinite number of digits chosen like this:
 - Choose the first fractional digit x_1 so that it is different from the first fractional digit of $f(1)$.
 - Choose the second fractional digit x_2 so that it is different from the second fractional digit of $f(2)$.
 - Choose the n^{th} fractional digit x_n so that it is different from the n^{th} fractional digit of $f(n)$.

The Impossible One: Reals

- Let f be a correspondence from $\mathbb{N}$ to $\mathbb{R}$. So each number $f(n)$ must be a real number.
- Now construct a number x , between 0 and 1, with an infinite number of digits chosen like this:
 - Choose the first fractional digit x_1 so that it is different from the first fractional digit of $f(1)$.
 - Choose the second fractional digit x_2 so that it is different from the second fractional digit of $f(2)$.
 - Choose the n^{th} fractional digit x_n so that it is different from the n^{th} fractional digit of $f(n)$.
- x is clearly a real number, but it clearly cannot be equal to $f(n)$ for any n . By contradiction, f is not a correspondence from $\mathbb{N}$ to $\mathbb{R}$.

The Impossible One: Reals

- Let f be a correspondence from $\mathbb{N}$ to $\mathbb{R}$. So each number $f(n)$ must be a real number.
- Now construct a number x , between 0 and 1, with an infinite number of digits chosen like this:
 - Choose the first fractional digit x_1 so that it is different from the first fractional digit of $f(1)$.
 - Choose the second fractional digit x_2 so that it is different from the second fractional digit of $f(2)$.
 - Choose the n^{th} fractional digit x_n so that it is different from the n^{th} fractional digit of $f(n)$.
- x is clearly a real number, but it clearly cannot be equal to $f(n)$ for any n . By contradiction, f is not a correspondence from $\mathbb{N}$ to $\mathbb{R}$.
- We write that $\mathbb{R}$ is **uncountably infinite**, or simply **uncountable**.

Back to Turing Machines

Some Observations

Countability of Turing Machines

- For any given alphabet we can do the same thing to it that we did to the rational numbers.
- In other words, the set of all strings over a given alphabet is countable.
- Any Turing machine **M** can be encoded into a string **<M>**.
- Therefore, the set of all Turing machines on a given alphabet is countable.

Some Observations

Countability of Turing Machines

- For any given alphabet we can do the same thing to it that we did to the rational numbers.
- In other words, the set of all strings over a given alphabet is countable.
- Any Turing machine **M** can be encoded into a string **<M>**.
- Therefore, the set of all Turing machines on a given alphabet is countable.

Infinite Binary Sequences

- An **infinite binary sequence** is an unending sequence of 0s and 1s.
- The set $\mathbb{B}$ of all of them is uncountable for the same reason the real numbers are.

Some Observations

Countability of Turing Machines

- For any given alphabet we can do the same thing to it that we did to the rational numbers.
- In other words, the set of all strings over a given alphabet is countable.
- Any Turing machine **M** can be encoded into a string **<M>**.
- Therefore, the set of all Turing machines on a given alphabet is countable.

Infinite Binary Sequences

- An **infinite binary sequence** is an unending sequence of 0s and 1s.
- The set $\mathbb{B}$ of all of them is uncountable for the same reason the real numbers are.

Languages and Infinite Binary Sequences

- Consider the set of all languages over a given alphabet.
- Remember that each string over that same alphabet can be a number.
- Consider a **characteristic sequence** of binary digits representing a language: the n th digit of this sequence is 1 if and only if the n th string is in the language.
- That's a correspondence between the languages over any alphabet and $\mathbb{B}$.
- The set of languages over a given alphabet is uncountable.

Some Observations

Countability of Turing Machines

- For any given alphabet we can do the same thing to it that we did to the rational numbers.
- In other words, the set of all strings over a given alphabet is countable.
- Any Turing machine **M** can be encoded into a string **<M>**.
- Therefore, the set of all Turing machines on a given alphabet is countable.

Infinite Binary Sequences

- An **infinite binary sequence** is an unending sequence of 0s and 1s.
- The set $\mathbb{B}$ of all of them is uncountable for the same reason the real numbers are.

Languages and Infinite Binary Sequences

- Consider the set of all languages over a given alphabet.
- Remember that each string over that same alphabet can be a number.
- Consider a **characteristic sequence** of binary digits representing a language: the n th digit of this sequence is 1 if and only if the n th string is in the language.
- That's a correspondence between the languages over any alphabet and $\mathbb{B}$.
- The set of languages over a given alphabet is uncountable.

Some Observations

Countability of Turing Machines

- For any given alphabet we can do the same thing to it that we did to the rational numbers.
- In other words, the set of all strings over a given alphabet is countable.
- Any Turing machine M can be encoded into a string $\langle M \rangle$.
- Therefore, the set of all Turing machines on a given alphabet is countable.

Infinite Binary Sequences

- An **infinite binary sequence** is an unending sequence of 0s and 1s.
- The set $\mathbb{B}$ of all of them is uncountable for the same reason the real numbers are.

Languages and Infinite Binary Sequences

- Consider the set of all languages over a given alphabet.
- Remember that each string over that same alphabet can be a number.
- Consider a **characteristic sequence** of binary digits representing a language: the n th digit of this sequence is 1 if and only if the n th string is in the language.
- That's a correspondence between the languages over any alphabet and $\mathbb{B}$.
- The set of languages over a given alphabet is uncountable.

There are Turing-unrecognizable languages.

An Unrecognizable Language

There Are Things We Still Can't Do!

An Undecidable Language: Setting Up

Let's see if we can decide the acceptance problem for Turing machines like we can for DFAs and CFGs.

$$\mathbf{A}_{\text{TM}} = \{ \langle \mathbf{M}, s \rangle \mid \mathbf{M} \text{ is a Turing machine that accepts } s \}.$$

An Undecidable Language: Setting Up

Let's see if we can decide the acceptance problem for Turing machines like we can for DFAs and CFGs.

$$\mathbf{A_{TM}} = \{ \langle \mathbf{M}, s \rangle \mid \mathbf{M} \text{ is a Turing machine that accepts } s \}.$$

Assume it's decidable. Given that, there exists a *decider* **H** for $\mathbf{A_{TM}}$ - that is, a Turing machine so that **H** run on $\langle \mathbf{M}, s \rangle$ accepts if **M** accepts *s* and rejects if **M** doesn't.

H($\langle \mathbf{M}, s \rangle$) accepts if **M** accepts *s* and rejects otherwise.

An Undecidable Language: Setting Up

Let's see if we can decide the acceptance problem for Turing machines like we can for DFAs and CFGs.

$$\mathbf{A}_{\text{TM}} = \{ \langle \mathbf{M}, s \rangle \mid \mathbf{M} \text{ is a Turing machine that accepts } s \}.$$

Assume it's decidable. Given that, there exists a *decider* $\mathbf{H}$ for $\mathbf{A}_{\text{TM}}$ - that is, a Turing machine so that $\mathbf{H}$ run on $\langle \mathbf{M}, s \rangle$ accepts if $\mathbf{M}$ accepts s and rejects if $\mathbf{M}$ doesn't.

$\mathbf{H}(\langle \mathbf{M}, s \rangle)$ accepts if $\mathbf{M}$ accepts s and rejects otherwise.

Now consider a new “*Devil's Advocate*” machine $\mathbf{D}$. This machine takes the decider, *runs it on the encoding of the original machine*, and *outputs the opposite result*.

$\mathbf{D}(\langle \mathbf{M} \rangle)$ accepts if $\mathbf{H}(\langle \mathbf{M}, \langle \mathbf{M} \rangle \rangle)$ rejects, and rejects if it accepts.

An Undecidable Language: Setting Up

Let's see if we can decide the acceptance problem for Turing machines like we can for DFAs and CFGs.

$$\mathbf{A}_{\text{TM}} = \{ \langle \mathbf{M}, s \rangle \mid \mathbf{M} \text{ is a Turing machine that accepts } s \}.$$

Assume it's decidable. Given that, there exists a *decider* $\mathbf{H}$ for $\mathbf{A}_{\text{TM}}$ - that is, a Turing machine so that $\mathbf{H}$ run on $\langle \mathbf{M}, s \rangle$ accepts if $\mathbf{M}$ accepts s and rejects if $\mathbf{M}$ doesn't.

$\mathbf{H}(\langle \mathbf{M}, s \rangle)$ accepts if $\mathbf{M}$ accepts s and rejects otherwise.

Now consider a new “*Devil's Advocate*” machine $\mathbf{D}$. This machine takes the decider, *runs it on the encoding of the original machine*, and *outputs the opposite result*.

$\mathbf{D}(\langle \mathbf{M} \rangle)$ accepts if $\mathbf{H}(\langle \mathbf{M}, \langle \mathbf{M} \rangle \rangle)$ rejects, and rejects if it accepts.

In other words,

$\mathbf{D}(\langle \mathbf{M} \rangle)$ accepts if $\mathbf{M}$ rejects $\langle \mathbf{M} \rangle$, and rejects if $\mathbf{M}$ accepts $\langle \mathbf{M} \rangle$.

An Undecidable Language: Closing the Trap

So far, we have:

$A_{TM} = \{ \langle M, s \rangle \mid M \text{ is a Turing machine that accepts } s \}.$

$H(\langle M, s \rangle)$ accepts if M accepts s and rejects otherwise.

$D(\langle M \rangle)$ accepts if $H(\langle M, \langle M \rangle \rangle)$ rejects, and rejects if it accepts.

$D(\langle M \rangle)$ accepts if M rejects $\langle M \rangle$, and rejects if M accepts $\langle M \rangle$.

An Undecidable Language: Closing the Trap

So far, we have:

$A_{TM} = \{ \langle M, s \rangle \mid M \text{ is a Turing machine that accepts } s \}.$

$H(\langle M, s \rangle)$ accepts if M accepts s and rejects otherwise.

$D(\langle M \rangle)$ accepts if $H(\langle M, \langle M \rangle \rangle)$ rejects, and rejects if it accepts.

$D(\langle M \rangle)$ accepts if M rejects $\langle M \rangle$, and rejects if M accepts $\langle M \rangle$.

Now, we try running D on itself.

$D(\langle D \rangle)$ accepts if D rejects $\langle D \rangle$, and rejects if D accepts $\langle D \rangle$.

This is as obvious a contradiction as you can get. A_{TM} is undecidable.

An Undecidable Language: Closing the Trap (with diagonalization)

- In case the proof isn't obvious, we will use the diagonalization technique to examine the behavior of the Turing Machines (H and D).
- Each row contains the TM $M_1, M_2, M_3, \dots$
- Each column is the descriptor $\langle M_1 \rangle, \langle M_2 \rangle, \dots$
- Each entire tells whether the machines accepts the input. (accept and reject)
- Let's makeup an example of certain machines that accepts and rejects the input.
- We can see the results of running H on the inputs.
- Notice M_3 does not accept input $\langle M_2 \rangle$
 - *That means H rejects input $\langle M_3, \langle M_2 \rangle \rangle$*

	$\langle M_1 \rangle$	$\langle M_2 \rangle$	$\langle M_3 \rangle$	$\langle M_4 \rangle$	...
M_1	accept	reject	accept	reject	
M_2	accept	accept	accept	accept	
M_3	reject	reject	reject	reject	...
M_4	accept	accept	reject	reject	
			...		

An Undecidable Language: Closing the Trap (with diagonalization)

- Let's add D to our table.
- From the assumption, H is a Turing Machine and so is D.
 - Occurs on lists M_1 , M_2 , and all Turing Machines

	$\langle M_1 \rangle$	$\langle M_2 \rangle$	$\langle M_3 \rangle$	$\langle M_4 \rangle$	...	$\langle D \rangle$	...
M_1	accept	reject	accept	reject	...	accept	...
M_2	accept	accept	accept	accept		accept	
M_3	reject	reject	reject	reject		reject	
M_4	accept	accept	reject	reject		accept	
...							
D	reject	reject	accept	accept	...	?	
...							

An Undecidable Language: Closing the Trap (with diagonalization)

- Let's add D to our table.
- From the assumption, H is a Turing Machine and so is D.
 - Occurs on lists M_1 , M_2 , and all Turing Machines
- Observe closely that the diagonal entries are the opposite of D.

	$\langle M_1 \rangle$	$\langle M_2 \rangle$	$\langle M_3 \rangle$	$\langle M_4 \rangle$	...	$\langle D \rangle$	...
M_1	<u>accept</u>	reject	accept	reject	...	accept	
M_2	accept	<u>accept</u>	accept	accept		accept	
M_3	reject	reject	<u>reject</u>	reject		reject	...
M_4	accept	accept	reject	<u>reject</u>		accept	
	...						
D	<u>reject</u>	<u>reject</u>	<u>accept</u>	<u>accept</u>	...	?	
	...						

An Undecidable Language: Closing the Trap (with diagonalization)

- Let's add D to our table.
- From the assumption, H is a Turing Machine and so is D.
 - Occurs on lists M_1 , M_2 , and all Turing Machines
- Observe closely that the diagonal entries are the opposite of D.
- The contradiction occurs at the '?' point.

	$\langle M_1 \rangle$	$\langle M_2 \rangle$	$\langle M_3 \rangle$	$\langle M_4 \rangle$	...	$\langle D \rangle$	...
M_1	<u>accept</u>	reject	accept	reject	...	accept	
M_2	accept	<u>accept</u>	accept	accept		accept	
M_3	reject	reject	<u>reject</u>	reject		reject	...
M_4	accept	accept	reject	<u>reject</u>		accept	
	...						
D	<u>reject</u>	<u>reject</u>	<u>accept</u>	<u>accept</u>	...	?	
	...						

Cleanup: An Unrecognizable Language

- Call a language **co-Turing recognizable** if its complement is recognizable.
 - A language that is decidable is obviously both Turing recognizable and co-Turing recognizable.
 - A language that's both Turing recognizable and co-Turing recognizable is almost as obviously decidable – just run the recognizer and co-recognizer in parallel, and return according to whichever one accepts first.
- So a language is decidable if and only if it is both recognizable and co-recognizable.
- A_{TM} is obviously *recognizable* – but we've already proven that it's undecidable. So $\overline{A_{TM}}$ is unrecognizable.



Acknowledgement

Some Notes and content come from Dr. Gerber, Dr. Hughes, and Mr. Guha's COT4210 class and the Sipser Textbook, *Introduction to the Theory of Computation*, 3rd ed., 2013